

```

# =====
# INTERNSHIP TASK 5
# Ensemble Learning, Model Optimization & ML Pipeline
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# -----
# 0. GENERATE SYNTHETIC DATASET (Fix for missing file)
# -----

X_raw, y_raw = make_classification(
    n_samples=500,
    n_features=8,
    n_informative=6,
    n_redundant=2,
    random_state=42
)

# Create a DataFrame with structured column names
columns = [f"Feature_{i+1}" for i in range(8)]
df_synthetic = pd.DataFrame(X_raw, columns=columns)
df_synthetic["Approved"] = y_raw

# Save the synthetic dataset to match the expected CSV name
df_synthetic.to_csv("classification_data.csv", index=False)

# -----
# 1. LOAD DATASET
# -----

df = pd.read_csv("classification_data.csv")

print("===== DATASET =====")
print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())

# -----
# 2. DATA INFORMATION
# -----

print("\n===== DATA INFORMATION =====")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

# -----
# 3. HANDLE MISSING VALUES
# -----

numeric_columns = df.select_dtypes(include=np.number).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

categorical_columns = df.select_dtypes(include="object").columns

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

# -----
# 4. ENCODE CATEGORICAL DATA
#

```

```

for col in df.select_dtypes(include="object").columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])

# -----
# 5. SEPARATE FEATURES AND TARGET
# -----

X = df.drop("Approved", axis=1)
y = df["Approved"]

print("\nFeatures:")
print(X.columns.tolist())

print("\nTarget:", y.name)

# -----
# 6. TRAIN-TEST SPLIT
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# =====
# 7. RANDOM FOREST
# =====

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)
rf_precision = precision_score(
    y_test, rf_pred, average="weighted", zero_division=0
)
rf_recall = recall_score(
    y_test, rf_pred, average="weighted", zero_division=0
)
rf_f1 = f1_score(
    y_test, rf_pred, average="weighted", zero_division=0
)

print("\n===== RANDOM FOREST =====")
print("Accuracy :", rf_accuracy)
print("Precision:", rf_precision)
print("Recall   :", rf_recall)
print("F1 Score :", rf_f1)

print("\nClassification Report:")
print(classification_report(y_test, rf_pred, zero_division=0))

# =====
# 8. GRADIENT BOOSTING
# =====

gb_model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)

gb_model.fit(X_train, y_train)

gb_pred = gb_model.predict(X_test)

gb_accuracy = accuracy_score(y_test, gb_pred)
gb_precision = precision_score(

```

```

    y_test, gb_pred, average="weighted", zero_division=0
)
gb_recall = recall_score(
    y_test, gb_pred, average="weighted", zero_division=0
)
gb_f1 = f1_score(
    y_test, gb_pred, average="weighted", zero_division=0
)

print("\n===== GRADIENT BOOSTING =====")
print("Accuracy :", gb_accuracy)
print("Precision:", gb_precision)
print("Recall   :", gb_recall)
print("F1 Score :", gb_f1)

print("\nClassification Report:")
print(classification_report(y_test, gb_pred, zero_division=0))

# =====
# 9. HYPERPARAMETER TUNING
# =====

print("\n===== HYPERPARAMETER TUNING =====")

param_grid = {
    "n_estimators": [50, 100],
    "max_depth": [None, 5, 10],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest Cross-Validation Accuracy:")
print(grid_search.best_score_)

# =====
# 10. EVALUATE TUNED MODEL
# =====

best_rf = grid_search.best_estimator_

tuned_pred = best_rf.predict(X_test)

tuned_accuracy = accuracy_score(y_test, tuned_pred)
tuned_precision = precision_score(
    y_test, tuned_pred, average="weighted", zero_division=0
)
tuned_recall = recall_score(
    y_test, tuned_pred, average="weighted", zero_division=0
)
tuned_f1 = f1_score(
    y_test, tuned_pred, average="weighted", zero_division=0
)

print("\n===== TUNED RANDOM FOREST =====")
print("Accuracy :", tuned_accuracy)
print("Precision:", tuned_precision)
print("Recall   :", tuned_recall)
print("F1 Score :", tuned_f1)

print("\nClassification Report:")
print(classification_report(y_test, tuned_pred, zero_division=0))

# =====
# 11. CROSS VALIDATION
# =====

print("\n===== CROSS VALIDATION =====")

cv_scores = cross_val_score(
    best_rf,

```

```

X,
y,
cv=5,
scoring="accuracy"
)

print("CV Scores:", cv_scores)
print("Mean CV Accuracy:", cv_scores.mean())
print("CV Standard Deviation:", cv_scores.std())

# =====
# 12. MODEL COMPARISON
# =====

results = pd.DataFrame({
    "Model": [
        "Random Forest",
        "Gradient Boosting",
        "Tuned Random Forest"
    ],
    "Accuracy": [
        rf_accuracy,
        gb_accuracy,
        tuned_accuracy
    ],
    "Precision": [
        rf_precision,
        gb_precision,
        tuned_precision
    ],
    "Recall": [
        rf_recall,
        gb_recall,
        tuned_recall
    ],
    "F1 Score": [
        rf_f1,
        gb_f1,
        tuned_f1
    ]
})

print("\n===== MODEL COMPARISON =====")
print(results.to_string(index=False))

# =====
# 13. CONFUSION MATRIX
# =====

cm = confusion_matrix(y_test, tuned_pred)

print("\n===== CONFUSION MATRIX =====")
print(cm)

plt.figure(figsize=(6, 5))
plt.imshow(cm)
plt.title("Tuned Random Forest - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()

for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.show()

# =====
# 14. FEATURE IMPORTANCE
# =====

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": best_rf.feature_importances_
})

importance = importance.sort_values(

```

```
        by="Importance",
        ascending=False
    )

print("\n===== FEATURE IMPORTANCE =====")
print(importance.to_string(index=False))

# =====
# 15. FEATURE IMPORTANCE GRAPH
# =====

plt.figure(figsize=(10, 6))

plt.barh(
    importance["Feature"],
    importance["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Random Forest Feature Importance")
plt.gca().invert_yaxis()

plt.show()

# =====
# 16. FINAL RESULT
# =====

best_model = results.loc[
    results["Accuracy"].idxmax(),
    "Model"
]

best_accuracy = results["Accuracy"].max()

print("\n=====")
print("      TASK 5 COMPLETED SUCCESSFULLY")
print("=====")
print("Best Model      :", best_model)
print("Best Accuracy:", round(best_accuracy, 4))

print("\nTopics Completed:")
print("1. Ensemble Learning")
print("2. Random Forest")
print("3. Gradient Boosting")
print("4. Hyperparameter Tuning")
print("5. Cross Validation")
print("6. Model Comparison")
print("7. Feature Importance")
print("8. ML Pipeline")
```


===== DATASET =====

Shape: (500, 9)

First 5 rows:

	Feature_1	Feature_2	Feature_3	Feature_4	Feature_5	Feature_6	\
0	0.311907	-2.098075	-1.046738	4.078113	-2.431152	-3.099275	
1	-0.250856	0.013947	2.248962	0.805360	0.378651	-0.588734	
2	-1.043991	-1.663572	-0.107572	1.473012	1.142945	1.069722	
3	-2.326774	-1.933848	-0.485261	-0.610549	-0.535475	-0.922035	
4	-0.955335	-3.698515	0.654567	3.956675	0.574079	0.512788	

	Feature_7	Feature_8	Approved
0	1.401672	-0.030395	0
1	-1.221643	-2.356418	1
2	-2.044641	-2.036647	1
3	-0.563406	2.130794	0
4	-2.862450	-2.584259	1

===== DATA INFORMATION =====

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 500 entries, 0 to 499

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Feature_1	500 non-null	float64
1	Feature_2	500 non-null	float64
2	Feature_3	500 non-null	float64
3	Feature_4	500 non-null	float64
4	Feature_5	500 non-null	float64
5	Feature_6	500 non-null	float64
6	Feature_7	500 non-null	float64
7	Feature_8	500 non-null	float64
8	Approved	500 non-null	int64

dtypes: float64(8), int64(1)

memory usage: 35.3 KB

None

Missing Values:

Feature_1	0
Feature_2	0
Feature_3	0
Feature_4	0
Feature_5	0
Feature_6	0
Feature_7	0
Feature_8	0
Approved	0

dtype: int64

Features:

['Feature_1', 'Feature_2', 'Feature_3', 'Feature_4', 'Feature_5', 'Feature_6', 'Feature_7', 'Feature_8']

Target: Approved

Training Data: (400, 8)

Testing Data: (100, 8)

===== RANDOM FOREST =====

Accuracy : 0.91

Precision: 0.9114813327980731

Recall : 0.91

F1 Score : 0.909918927034331

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.94	0.91	50
1	0.94	0.88	0.91	50
accuracy			0.91	100
macro avg	0.91	0.91	0.91	100
weighted avg	0.91	0.91	0.91	100

===== GRADIENT BOOSTING =====

Accuracy : 0.91

Precision: 0.9141414141414141

Recall : 0.91

F1 Score : 0.9097744360902256

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.96	0.91	50
1	0.96	0.86	0.91	50
accuracy			0.91	100
macro avg	0.91	0.91	0.91	100
weighted avg	0.91	0.91	0.91	100

```
===== HYPERPARAMETER TUNING =====
Best Parameters:
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}

Best Cross-Validation Accuracy:
0.9

===== TUNED RANDOM FOREST =====
Accuracy : 0.91
Precision: 0.9114813327980731
Recall   : 0.91
F1 Score : 0.909918927034331

Classification Report:
      precision    recall  f1-score   support

     0       0.89       0.94       0.91         50
     1       0.94       0.88       0.91         50

 accuracy          0.91         0.91         0.91        100
  macro avg       0.91       0.91       0.91        100
 weighted avg     0.91       0.91       0.91        100
```